2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA) | 979-8-3503-2658-1/24/$31.00 ©2024 IEEE | DOI: 10.1109/ISCA59077.2024.00071

Flagger: Cooperative Acceleration for Large-Scale Cross-Silo Federated Learning Aggregation

Xiurui Pan*, Yuda An*, Shengwen Liang†, Bo Mao‡,
Mingzhe Zhang§, Qiao Li‡, Myoungsoo Jung¶, and Jie Zhang*
Computer Hardware and System Evolution Laboratory
Peking University*, Institute of Computing Technology, Chinese Academy of Sciences†, Xiamen University‡
Institute of Information Engineering, Chinese Academy of Sciences§, KAIST and Panmnesia¶
https://www.chaselab.wiki

***Abstract*—Cross-silo federated learning (FL) leverages homomorphic encryption (HE) to obscure the model updates from the clients. However, HE poses the challenges of complex cryptographic computations and inflated ciphertext sizes. As cross-silo FL scales to accommodate larger models and more clients, the overheads of HE can overwhelm a CPU-centric aggregator architecture, including excessive network traffic, enormous data volume, intricate computations, and redundant data movements. Tackling these issues, we propose *Flagger*, an efficient and high-performance FL aggregator. Flagger meticulously integrates the data processing unit (DPU) with computational storage drives (CSD), employing these two distinct near-data processing (NDP) accelerators as a holistic architecture to collaboratively enhance FL aggregation. With the delicate delegation of complex FL aggregation tasks, we build Flagger-DPU and Flagger-CSD to exploit both in-network and in-storage HE acceleration to streamline FL aggregation. We also implement Flagger-Runtime, a dedicated software layer, to coordinate NDP accelerators and enable direct peer-to-peer data exchanges, markedly reducing data migration burdens. Our evaluation results reveal that Flagger expedites the aggregation in FL training iterations by 436% on average, compared with traditional CPU-centric aggregators.**

I. Introduction

As various domains of machine learning, such as computer vision and natural language processing, increasingly rely on high-quality models, the demand for large volumes of training data is continually rising [104]. However, these data are geo-distributed across diverse data repositories or "silos", such as governments, hospitals, and financial companies [9], [71], [84]. Unfortunately, strict restrictions on data-sharing are usually enforced among these entities due to escalating concerns such as data confidentiality and legal regulations [2]–[4].

A promising solution is cross-silo *federated learning* (FL), which facilitates collaborative model training among multiple data silos without compromising the data privacy [39], [55]. In this paradigm, the data silos (or "clients") train models locally, while only the intermediate results (e.g., parameters and gradient updates) are shared with an auxiliary third-party server (or "aggregator") through wide-area networks (WAN). Aggregator collates these results to produce a global model. To enhance privacy, industrial FL frameworks employ homomorphic encryption (HE) to obscure local updates despite its significant drawbacks. It increases computational complexity and training time by over 100× and expands data volume by over 150× due to the ciphertext size inflation [11], [54], [101].

While several previous studies have focused on mitigating the HE-related overheads for FL clients to accelerate local computation [14], [21], [87], [97], [103], the impact on the global aggregation in cross-silo FL has been less explored. As FL scales up to accommodate larger models [82] and more clients [32], the global aggregation process becomes a performance-limiting factor. For instance, in our experiments employing 32 or 64 clients with abundant resources to train six machine learning models, the aggregation process consumed up to 82.75% of the total training time for a single cross-silo FL training iteration (cf. Figure 3a). This significant overhead incurred by aggregation arises from four main factors.

- *Network*: Transmitting large volume of ciphertexts through low-speed WAN between the aggregator and numerous clients contributes to a considerable latency for the overall iteration (cf. Section III-B).
- *Computation*: The computational capability of the aggregator is not only inadequate but also underutilized for complex cryptographic operations, accounting for 45.83% of the aggregator time, as seen under `Computation` in Figure 3b.
- *Storage*: Although incorporating high-capacity solid state drives (SSDs) in the aggregator can help mitigate the memory pressure resulting from the expanded ciphertexts, the I/O access delay inherent in SSDs incurs 49.69% longer latency, reflected as `SSD` in Figure 3b.
- *Data path*: Frequent transmission of inflated ciphertexts through the aggregator's components (NIC, memory, accelerator, SSD) results in an elongated data path and multiple data replications, causing a 70.31% extension of the aggregation latency (cf. Figure 3b for `Internal IO`).

Near data processing (NDP) is an effective approach for mitigating performance limitations of existing architectures by facilitating computations close to data sources [13], [27], [83]. Specifically, the novel *computational storage drive* (CSD) performs computations close to large data volumes, thus taking advantage of the high internal bandwidth of flash channels [50], [52], [58]. Conversely, a *data processing unit* (DPU) provides ample computational resources near the network port for handling streamed data with minimal memory [16], [85],

979-8-3503-2658-1/24/$31.00 ©2024 IEEE
DOI 10.1109/ISCA59077.2024.00071

[100]. However, none of the existing near-data accelerators efficiently address the challenges of FL aggregation. Specifically, typical CSDs lack the required computing power for intricate homomorphic operations in FL. On the other hand, the DPUs only support data processing in a streaming manner, whereas FL aggregation cannot be conducted until complete data collection. Furthermore, simply assembling existing DPUs and CSDs is also infeasible due to the lack of direct datapath and unified scheduling mechanisms among different accelerators.

To address these hurdles, we introduce *Flagger*, a collaborative, streamlined architecture with both DPU and CSD as distinct near-data accelerators for efficient large-scale cross-silo FL aggregation [1]. Specifically, Flagger integrates three main components: i) Flagger-DPU, ii) Flagger-CSD, and iii) host-side Flagger-Runtime. To reduce the computing overhead of complex homomorphic operations, we harness the computing power of both Flagger-DPU and Flagger-CSD. These accelerators work in unison, with the coordination of Flagger-Runtime, to speed up different phases of aggregation tasks based on their unique computation and I/O features. With delicate task delegation, Flagger exploits both in-network and in-storage processing techniques, which efficiently mitigate the overheads incurred by network transmission and storage accesses. The peer-to-peer DMA capability amongst the accelerators further streamlines the data path by eliminating the reliance on the host for redundant data copies. Our evaluation shows that, compared with a traditional CPU-centric FL aggregator with adequate resources, Flagger expedites the aggregation process and overall FL iteration by 436% and 277%, respectively.

The main **contributions** of this work are as follows:

• *Cooperative DPU-CSD architecture for FL aggregation:* We argue that simply employing any type of existing accelerator in the FL aggregator cannot completely address the various challenges of the traditional computing system (i.e., computation, storage, network, and data path). To mitigate these overheads, we propose a novel approach that synergizes both DPU and CSD as near-data processing accelerators to minimize data movement overheads. We further shorten the tedious datapath by enabling direct data transmission between the DPU and CSD via peer-to-peer DMA, eliminating the dependence on the host for data movement. Considering the unique computation and I/O features of DPU and CSD, we distribute the aggregation task into distinct but tightly coupled phases. A unified data management and scheduling mechanism is performed between both DPU and CSD, which dynamically assigns the aggregation phases to DPU and CSD based on their exclusive attributes and real-time loads.

• *Efficient in-network HE acceleration with Flagger-DPU:* Conventional DPUs fall short in exploiting the in-network computing potential to mitigate overheads incurred by the lengthy ciphertext transmission during FL aggregation. Flagger overcomes this limitation by strategically offloading the compute-intensive *pre/post-processing* tasks to Flagger-DPU, thereby capitalizing the extensive transmission time across WAN. Specifically, Flagger integrates high-performance HE computing engines into the hardware-based TCP/IP stack on Flagger-DPU, allowing Flagger to perform intricate HE operations on ciphertexts embedded in TCP packets directly.This efficient in-network offloading also markedly alleviates the computing burden while aggregating ciphertexts in Flagger-CSD, which significantly boosts the overall throughput.

• *Lightweight in-storage ciphertext aggregation with Flagger-CSD:* Existing CSDs struggle with efficient large-scale ciphertext aggregation due to burdensome homomorphic computations, complex control, and long flash access latencies. Flagger addresses these inefficiencies by only assigning the IO-intensive *combination* tasks to Flagger-CSD, which exhibit substantially reduced computational intensity with *pre-/post-processing* operations offloaded by Flagger-DPU. Flagger-CSD further meticulously exploits the channel-level parallelism and constructs the compute-IO pipeline to prevent the aggregation from being stalled by accessing flash chips. Moreover, the autonomous ciphertext management by Flagger-CSD's FTL further obviates the reliance on intricate control commands from host, thereby further reducing the complexity and improving the throughput of in-storage aggregation.

[1]Flagger is available at https://github.com/ChaseLab-PKU/Flagger.

II. Background

A. Federated Learning

Cross-silo Federated Learning (FL). FL is a collaborative effort in which geographically distributed clients work together to train a shared machine learning model [32], [39], [55]. The linchpin of this method is its commitment to privacy, as the raw training data remain unseen by others. Instead, model updates (e.g., gradients) are collected by a third-party aggregator to build a global model. In cross-silo FL, the clients are data silos, or *entities* with access to data, such as hospitals, banks, or government institutions [9], [71], [84]. These entities often have considerable computational resources and stable network access, allowing them to train larger, higher-quality models. However, they also require stricter security measures [2]–[4] to ensure no data leakage through model updates. In this work, we assume that the aggregator is honest-but-curious, a widely adopted threat model in existing FL research [37], [101], which means the aggregator obeys the FL protocols but tries to infer the sensitive data from model updates of clients. To fulfill the security requirements, industrial FL frameworks, like FATE [7], use homomorphic encryption (HE) as a standard to encrypt model updates.

Homomorphic Encryption (HE). HE is a form of encryption technique that allows direct arithmetic operations (like addition and multiplication) on ciphertexts. Depending on the type of supported computation, HE can be categorized into partial homomorphic encryption (PHE), somewhat homomorphic encryption (SWHE), and fully homomorphic encryption (FHE). FHE supports an unlimited number of addition and multiplication, which is the most powerful but incurs unacceptable overheads [45]. PHE, on the other hand, only supports a certain type of calculation. Paillier scheme [65], as a kind of PHE, is extensively used in cross-silo FL. Paillier supports additive

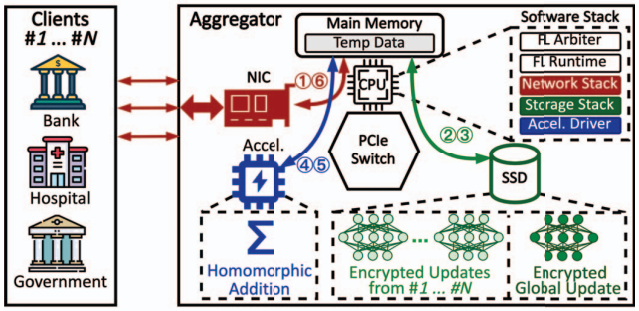


Fig. 1: A conventional FL system and the aggregator internals.

calculation without precision loss, eliminating the burdensome key-switching, basis conversion, or bootstrapping operations in FHE for multiplication. Paillier-based HE aligns well with FL requirements, as it allows the aggregation of encrypted model updates through homomorphic addition without compromising the precision of the original model [54]. Considering the tradeoff between security overheads and performance, the most widely adopted key size in Paillier HE encryption for FL applications is 1024 bits [66]. Note that Paillier-based homomorphic additions still introduce significant overheads in two aspects compared with direct aggregation on plaintexts. First, they involve extremely time-intensive long integer modulo operations. Second, the public key size determines the ciphertext size, inflating an FP16 number in model gradients into an integer twice the size of the public key (e.g., 2048 bits) [37], [101]. This results in considerable data volume inflation.

FL training details. In cross-silo FL, clients share a common feature space with distinct sample spaces. FL allows clients to train a common model in collaboration, thereby enhancing the model's generalization capability by increasing the training sample size. A simplified t^{th} FL training iteration, involving multiple clients and an aggregator, proceeds as follows:

1) All clients negotiate about shared private and public encryption keys: key_{priv} and key_{pub}.
2) Client i utilizes its own data to train its local model $\boldsymbol{w_i^t}$, generates the local updates such as gradients $\boldsymbol{g_i^t}$, and encrypts $\boldsymbol{g_i^t}$ as $[[\boldsymbol{g_i^t}]]$ with key_{pub} using additively homomorphic encryption (e.g., Paillier).
3) Clients transmit their encrypted model updates $[[\boldsymbol{g_i^t}]]$ to the centralized aggregator.
4) The aggregator adds the encrypted updates into the global model updates $\frac{\sum_i [[\boldsymbol{g_i^t}]]}{n}$, where n represents the client count. The use of additively homomorphic encryption allows for direct aggregation on the ciphertext.
5) The aggregator then distributes the global updates $\frac{\sum_i [[\boldsymbol{g_i^t}]]}{n}$ to all clients.
6) Upon receiving the global updates, each client decrypts them with key_{priv} and updates its local model to $\boldsymbol{w_i^{t+1}}$.

This FL training approach ensures all clients end with a uniformly trained model. Note that FL training in production can be more complex to enhance its stability and fairness. For instance, as clients may drop out during updating due to network issues, the aggregator will exclude entire model updates from incomplete clients [72]. Besides, FL adopts a reward mechanism to incentivize clients to update data as soon as possible [26]. Once all the uploads are completed, the aggregator attaches different weights to the model updates of clients based on the final uploading time ratio. These mechanisms require the aggregator to store all the model updates before initiating its aggregation, which prevents the aggregator from adopting a streaming-based aggregation approach (i.e., calculating the partial sum right after the ciphertexts are received instead of storing them in the SSD) to reduce memory footprint.

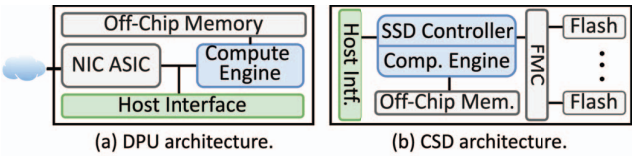


Fig. 2: Typical architectures of two near-data accelerators.

Typical FL system architecture. Figure 1 presents a typical FL system for the training process above [55], [56]. The aggregator primarily comprises three distinct components: a host (CPU and main memory), a network interface card (NIC), and solid-state drives (SSDs). Some also employ a dedicated accelerator to expedite the compute-intensive homomorphic computations. These elements are interconnected via the PCIe switch. The host CPU executes the software stack, encompassing an FL arbiter program, an FL runtime, I/O stacks, and accelerator drivers to manage the entire FL process, process FL tasks, manage data for network/storage accesses (i.e., NIC and SSD), and interact with the accelerators, respectively.

To aggregate encrypted local model updates, the following steps are executed. When local model updates arrive as network packets from WAN (e.g., the Internet), they are moved from the NIC to a temporary buffer in main memory (①), processed through the network stack by the CPU, and transmitted through storage stack to be stored as ciphertexts on the SSD (②). After the SSD collects all ciphertexts, they are reloaded into main memory (③) and forwarded to the accelerator for HE-based aggregation (④). Once aggregated, the results are stored back in main memory (⑤). Given the accelerator's limited off-chip memory, model updates are divided and processed iteratively in multiple batches (cf. Section III-B). Finally, the aggregated global updates are transferred from main memory to the NIC (⑥) and dispatched to clients over the network.

Note that all data traffic and aggregation tasks converge at the aggregator in a cross-silo FL system. Nevertheless, the traditional aggregator architecture struggles to effectively manage these tasks, leading to significant performance bottlenecks in the FL system (cf. Section III for quantitative elaboration).

B. Near-Data Processing

Data processing unit (DPU). A DPU, also known as a SmartNIC, typically serves as an extension of the NIC [22], [60]. It chiefly takes over portions of tasks performed by the host, thus significantly reducing CPU overheads. Although

DPU architectures vary widely to meet diverse goals, they generally consist of a NIC ASIC, compute engines (FPGA, ASIC, or ARM cores), off-chip memory (DRAM or HBM), and a host interface [17], [53], as shown in Figure 2(a). The NIC ASIC serves as a typical NIC, while the compute engine performs in-network acceleration. From the DPU's perspective, positioning computing engines close to the network port is beneficial. Given that network traffic is usually streamed from the network port, the DPU is optimally suited for real-time, stream-based near-data processing. However, the DPU's memory capacity is limited (4∼16 GB) [17], [53], restricting its ability to store large amounts of data. Consequently, it is not suited well for executing large-scale I/O-intensive tasks like aggregating inflated ciphertexts.

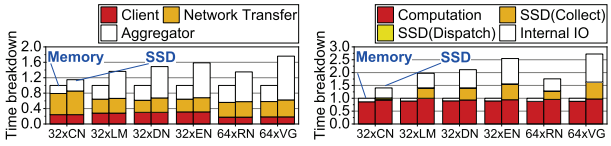
Computational storage drive (CSD). Conversely, a CSD is a distinct type of storage device with compute engines (e.g., ARM cores or an FPGA) [43], [47], [69]. A typical architecture of CSD is shown in Figure 2(b), which contains a host interface, an embedded processor serving as the SSD controller, compute engines, off-chip memory, and NAND flash chips. To access data in the SSD, the logical addresses of host memory requests are mapped to physical addresses by the flash translation layer (FTL) via the SSD controller, which then commands the flash memory controller (FMC) to access data through parallel flash channels. Considering the inherent non-trivial latency in flash access [46], [90], off-chip memory usually acts as a buffer for data in the flash. Unlike traditional CPU-centric architectures that require data to be transferred between the SSD and the host, the CSD performs calculations directly on a large amount of data stored in NAND flash. It can, therefore, leverage the high internal bandwidth of flash channels and eliminate the overheads imposed by unnecessary data transfers. However, due to the power budget and cost considerations, CSDs typically integrate only wimpy compute engines (e.g., limited ARM cores or low-end FPGA chips) [49], [69], which are insufficient for complex tasks such as homomorphic computations in FL and necessitate more capable systems for such demands.

III. Preliminary Study of Aggregators

A. Aggregation Bottleneck in Federated Learning

Data silos in cross-silo FL typically possess ample computing resources and high-speed network connections [59], [103], which significantly diminish computing overheads for the clients. Conversely, we observe that although the aggregator is equipped with ample resources as well, the aggregation inevitably becomes the primary bottleneck when scaling FL to accommodate larger models and more clients [32], [57].

To gain a better understanding of this situation, we analyze the execution time breakdown of six widely adopted cross-silo FL applications. Both the clients and the aggregator are equipped with abundant computational resources and network bandwidth, conforming real cross-silo setups (cf. Section VII-A for details). To prevent the aggregator from out-of-memory issues due to large ciphertext volume, we propose two storage configurations: using host `Memory` for all ciphertexts as the ideal case and employing `SSD`s for storage.



(a) Total time breakdown. (b) Aggregation time breakdown.

Fig. 3: Execution time breakdown in an FL iteration.

Figure 3a displays the breakdown of execution time during an FL training iteration, with `SSD` results normalized to `Memory`. The time consumed by the aggregation predominates the execution time in all six applications. Specifically, aggregation (`Network Transfer` and `Aggregator`) averages 74.98% of the total time in `Memory` configurations. Furthermore, if the aggregator has to employ SSDs as a storage backend, the aggregation time increases by an average of 113.83%, consuming 82.75% of the iteration time. We exclude the clients' gradient computation time, which is under 1% as reported by existing literature [37], [101].

B. Analysis of Overheads and Challenges

The performance penalties observed in the aggregation process can be attributed primarily to four significant factors: network, computation, storage, and data path.

Network: As shown in Figure 3a, network transmission contributes significantly to the total duration of end-to-end FL training iteration, accounting for 38.96% in the `Memory` scenario and 29.64% in the `Storage`, on average. Unlike the high-speed network links within a data center (e.g., rack-level RDMA), cross-silo FL relies on slower WAN and TCP/IP protocols to connect geo-distributed clients to the central aggregator. This leads to a marked increase in the time required for the aggregator to collect a substantial volume of ciphertexts from clients. Additionally, due to the heterogeneity in client capabilities, the time taken to upload model updates can vary widely, further prolonging the overall upload duration [15]. Nevertheless, although prior works propose various accelerators, including DPUs with in-network acceleration capabilities, these devices have to remain idle during the lengthy uploading process. This leads to significant resource underutilization (cf. Figure 9) and hinders the FL aggregation process.

Computation: Figure 3b further presents the overhead breakdown for the `aggregator` across various FL applications. Despite that the throughput for homomorphic addition by the aggregator (4.82MOPs) significantly outperforms that of homomorphic encryption by clients (134.4KOPs), the computing overhead of the aggregator (`Computation` in Figure 3b) still accounts for an average of 45.83% of the total overhead in `Storage`. This inefficiency stems from both under-utilized computing resources of the accelerator and the colossal volume of ciphertexts from numerous clients (cf. Section VII). Therefore, how to efficiently speed up the aggregation process under HE has become an urgent challenge.

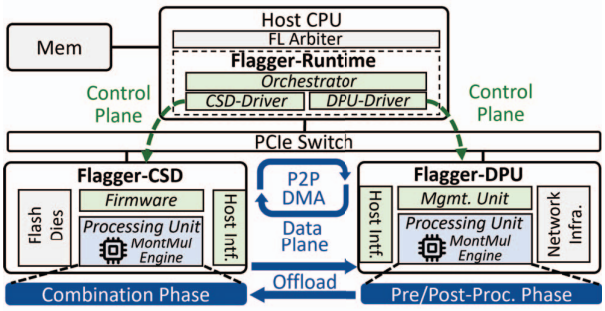


Fig. 4: Flagger overview.

Storage: Collecting inflated ciphertexts from numerous clients results in overwhelming data storage in the aggregator. During a single FL training iteration, we measured 0.75∼121.47 GB memory footprints in the aggregator for the six FL applications (cf. Table II), respectively. As larger models and more clients become suitable for FL [67], [82], the memory footprint could even reach hundreds of gigabytes. This necessitates the aggregator's reliance on SSDs to store these ciphertexts. However, compared to main memory, the longer latency and lower bandwidth of SSDs [19] could induce a substantial performance penalty in the traditional aggregator architecture. Figure 3b also showcases that employing SSDs to gather and dispatch model updates can increase the aggregator's time cost by 49.69% on average, compared to the `Memory` scenario. The overhead encompasses `SSD(Collect)` (40.96%), `SSD(Dispatch)` (1.75%), and extended `Computation` time (6.98%). Thus, identifying more efficient strategies to reduce the performance penalty caused by SSDs is crucial.

Data path: As outlined in Section II-A, the conventional aggregator architecture utilizes distinct hardware components for data storing and processing. However, the absence of a direct datapath between components requires coordination via the host, leading to redundant data movement between devices and the host with significant overheads. Specifically, extensive ciphertexts collected through the NIC must initially be copied to the host memory buffer before being stored on the SSD. Worse still, if the accelerator intends to aggregate ciphertexts, it cannot directly fetch data on the SSD either; instead, the data must be moved to the host memory buffer first. We find that these repeated data copies can extend the aggregation time cost by 58.95% on average (cf. `Internal IO` in Figure 3b). Hence, it is imperative to find ways to shorten this extended data path, thereby improving the aggregator's performance.

IV. Flagger Overview

Motivated by prior analysis, we present *Flagger*, a collaborative, streamlined architecture with heterogeneous near-data accelerators for efficient large-scale cross-silo FL aggregation, as depicted in Figure 4. Flagger is primarily comprised of the following key components:

- *Flagger-DPU* (cf. Section V-B), which handles network stack and facilitates in-network HE processing, thereby mitigating penalties from HE and WAN transmission.
- *Flagger-CSD* (cf. Section V-C), offering expansive storage and efficient in-storage aggregation capabilities for secure and flexible handling of client ciphertexts, thereby reducing extra data copies and improving throughput.
- *Flagger-Runtime* (cf. Section V-D), orchestrating control tasks between Flagger-DPU and Flagger-CSD, and interfacing with the FL arbiter for seamless operation and coordination within the architecture.

Equipped with compatible near-data computing engines, both Flagger-DPU and Flagger-CSD jointly accelerate homomorphic computations for aggregation, addressing the computation, storage, and communication bottlenecks inherent in traditional aggregator architectures.

Computation: To effectively accelerate the compute-intensive FL aggregation, we devised a high-performance computing engine with the Montgomery modular multiplication (*MontMul* for abbreviation) algorithm [10], [25], [97]. We seamlessly integrated the MontMul engine into both Flagger-DPU and Flagger-CSD, enabling the exploitation of NDP capabilities and cooperative execution to expedite the aggregation process.

The primary challenge remains how to exploit the potential of heterogeneous near-data accelerators by judiciously distributing computation tasks between the Flagger-DPU and Flagger-CSD, based on their distinct computation and I/O features. To surmount this, we partition the aggregation computation into three different phases. The first and last phases, focusing on data preparation, are allocated to Flagger-DPU to capitalize on its streamlined processing capability of network packets containing ciphertexts. The second phase, involving the actual aggregation of model updates, awaits the availability of all client ciphertexts at the aggregator-side storage. As this phase involves a substantial volume of ciphertexts stored on flash chips, we allocate the intensive combining process to Flagger-CSD, thereby harnessing in-storage aggregation and significantly reducing data-movement overhead.

Network: To mitigate the performance penalty from extended ciphertext transmission over WAN, we overlap the MontMul operations in the aggregation computation with network transmission by leveraging Flagger-DPU's in-network acceleration capability. In the pursuit of an appropriate DPU architecture, we discovered that DPUs with ARM cores (e.g., BlueField [53]), even enhanced with cutting-edge network acceleration techniques such as DPDK [35], fall short in meeting the demands of cross-silo FL aggregation, which entails high-speed TCP/IP networking coupled with intricate homomorphic computations [42], [83]. To overcome these challenges, we selected an FPGA-based approach for Flagger-DPU. To enable high-speed in-network homomorphic computation within the complex TCP/IP stack, we offloaded the entire hardware-based network stack to Flagger-DPU with optimizations for FL aggregation. Furthermore, we tailored the network stack to integrate directly with MontMul engines, enabling Flagger-DPU to process TCP packets containing ciphertexts without the need for redundant data copying in the DPU memory.

Storage: Prior studies have demonstrated that the aggregated I/O bandwidth of SSD internals is 1∼6 times higher than

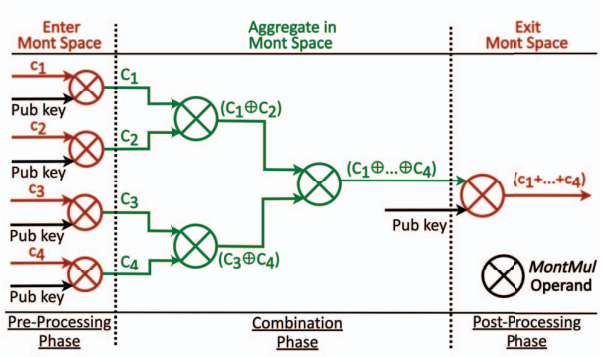


Fig. 5: Aggregation with the Montgomery algorithm.

that of the PCIe lanes [58], [102]. Taking a cue from this, Flagger-CSD addresses the performance penalty incurred by slow storage-I/O interface by leveraging the compute engine within the CSD to aggregate ciphertexts residing on flash chips. We meticulously constructed the I/O-compute pipeline to ensure the MontMul engine within Flagger-CSD is fully loaded, avoiding idle cycles and maximizing efficiency. To prevent the slow flash accesses from sitting on the critical path [90], intermediate results from the aggregation process are placed in the internal memory buffer, from which the compute engines can efficiently access data stored both in memory and on flash chips. Moreover, to streamline the aggregation process, Flagger-CSD directly manages the data layout via the FTL and circumvents the involvement of host-side filesystem. This design enables Flagger-CSD to autonomously complete the aggregation task without host intervention, simplifying the operation and reducing complexity.

Coordination: A salient drawback of the traditional aggregator architecture is its significant reliance on the host's main memory as a buffer for data transfer between different devices. To eliminate redundant data movements within Flagger, we employ peer-to-peer DMA transmission of ciphertexts between Flagger-DPU and Flagger-CSD, thereby completely bypassing the CPU and main memory. In terms of control, the Flagger-Runtime orchestrates the data transfers and dynamically offloads various task phases between Flagger-DPU and Flagger-CSD based on their real-time loads. In support of the coordination, we further adopt a unified data management mechanism on ciphertexts in each Flagger component to streamline the data transfer and task delegation. This coordination ensures efficient and flexible resource utilization, thereby enhancing the overall performance of the aggregation process.

V. Key Designs in Flagger

A. MontMul Engines

In the Paillier-based HE scheme, long-integer modulo multiplication, which serves as the core operand of homomorphic addition, is the primary source of its high computational complexity. A recent optimization technique, *Montgomery algorithm* [10], [97], [103], sheds light on this problem. Figure 5 delineates the key steps involved in aggregating four ciphertexts (c_1, c_2, c_3, c_4) via the Montgomery algorithm. Prior

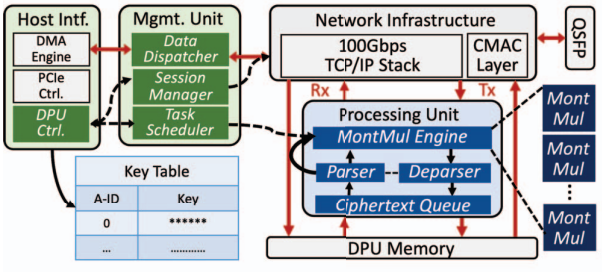


Fig. 6: Flagger-DPU architecture.

to initiation of the aggregation process, all ciphertexts must be transformed into the Montgomery space through Montgomery modulo multiplication (the `MontMul` operand). Within this space, the cumbersome long integer modulo multiplication is replaced by more efficient short integer multiplication, thereby streamlining the process of homomorphic additions.

Although Montgomery was not initially designed for near-data processing, it proves ideal for in-network and in-storage acceleration of FL aggregation with homomorphic encryption. As Flagger aims to assign different aggregation tasks to distinct near-data accelerators, the whole aggregation is thereby segmented into three phases based on the Montgomery algorithm: the *pre-processing* phase, where ciphertexts are transformed into Montgomery space; the *combination* phase, where ciphertexts are aggregated; and the *post-processing* phase, where the aggregated result is converted back. Notably, the same MontMul operation is employed throughout all three phases, which prompted us to equip the MontMul engine to optimize performance across different stages of the aggregation process (cf. Section V-D). Given that the pre-processing phase requires only a single ciphertext and the public key, it is well-suited for execution on a per-ciphertext basis. Therefore, we assign this phase to Flagger-DPU, which can be executed concurrently with ciphertext transmission from the network. This strategy ensures that all ciphertexts arriving at Flagger-CSD are already prepared in Montgomery space, obviating the need for additional conversions into the Montgomery space during the combining phase, thereby significantly reducing the computing burden. Similarly, the post-processing phase is also delegated to Flagger-DPU when dispatching the aggregated ciphertexts, which further reduces unnecessary data movements through in-network HE computation.

B. Flagger-DPU

Flagger-DPU features in-network HE computation of ciphertexts within the TCP/IP stack to streamline the aggregation process. Figure 6 depicts the Flagger-DPU architecture, comprising a hardware-based network infrastructure, a processing unit for in-network HE processing, a management unit for DPU control, and a host interface to interact with the host.

Processing network traffic. Primarily, the network infrastructure encompasses a CMAC layer [94] and a comprehensive TCP/IP stack, which is connected to the QSFP port to ensure rapid data transmission. The TCP/IP stack handles TCP/IP packets at a 100Gbps line rate to support high-throughput

WAN communication with geo-distributed clients. Integrating a complete network infrastructure in Flagger-DPU avoids the host involvement for network processing, thereby enabling efficient peer-to-peer data transfers to Flagger-CSD.

To further enable in-network HE processing within the TCP/IP stack, Flagger-DPU is meticulously designed to handle per-packet processing of ciphertexts, avoiding fragmentation issues caused by TCP segmentation [68]. To maintain the integrity of ciphertexts, each TCP packet is mandated to contain complete ciphertexts, precluding the possibility of a single ciphertext being split across multiple packets. Specifically, the maximum segmentation size (MSS) in a TCP packet for accommodating ciphertexts is 1460 B [68]. With the size of a ciphertext being 256 B in 1024-bit Paillier HE, each packet can encapsulate a maximum of five ciphertexts, yielding 1280 B of effective payload data and leaving the other bytes for padding. Flagger dictates that clients tag each ciphertext with IDs and transmit them sequentially, thus obviating the inner-packet ID recording of ciphertexts and simplifying the management. Despite a slight reduction in network throughput due to the padding to match the MSS, this method significantly improves the efficiency of the processing unit, allowing for online computation of received packets without requiring complex TCP packet reassembling or reordering [64].

Computing ciphertext packets. The Flagger-DPU's processing unit sits between the network infrastructure's data path and memory, thereby enabling online pre- and post-processing of packets. To avoid unnecessary computation due to TCP retransmissions, we pre-process packets as they're read from the TCP Rx buffer rather than during their writing to it. This approach stems from the fact that retransmission leads to multiple writes but only a single read to the Rx buffer for incast TCP traffic [77]. Similarly, the processing unit *post-processes* ciphertexts where packets are written to the Tx buffer.

Let us consider pre-processing as an illustrative example to demonstrate the workflow of the processing unit. During pre-processing, incoming Rx packets are queued in the processing unit, awaiting computation. Once the MontMul engine becomes available, the incoming packet is parsed for its IP address, *ACT-ID* (a unified identifier among different components of Flagger, details of which is discussed in Section V-C), and ciphertexts, which are then split and individually processed through MontMul units. After computation, the *deparser* reassembles the packets to route them back into the TCP/IP stack. This process not only transforms the ciphertext payload into Montgomery space for aggregation but also maintains the TCP packet structure for further network processing.

DPU control. Flagger-DPU's primary roles include client communication, network packet processing, and synchronization with Flagger-CSD, controlled by its on-chip management unit through a session manager, task scheduler, and data dispatcher, respectively (Figure 6). Specifically, the session manager ensures stable client connections via WAN and receives commands from the host-side Flagger-Runtime via the host interface (cf. Section V-D). To dynamically allocate network packets to MontMul units in the processing unit

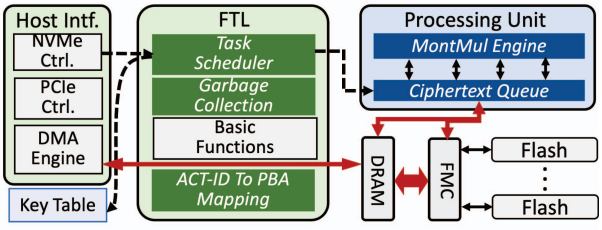


Fig. 7: Flagger-CSD architecture.

based on current load and demands, the task scheduler monitors the available resources within MontMul engine, which is essential for handling the variable network traffic with fluctuating incoming data volume and rate. Meanwhile, the data dispatcher manages data-plane tasks like transmitting ciphertexts to Flagger-CSD, utilizing addresses derived from ACT-ID for initiating DMA transactions through Flagger-Runtime. Flagger-DPU thereby provides application-level data streams to Flagger-CSD rather than the low-level packetized data abstraction offered by a traditional NIC, which eliminates the involvement of host I/O stack to process network packets.

C. Flagger-CSD

Figure 7 illustrates the Flagger-CSD architecture, which is devised to efficiently manage and aggregate ciphertexts in large-scale cross-silo FL. It communicates with the host and Flagger-DPU through the host interface, manages CSD through the FTL similarly to traditional SSDs, and performs aggregation computation tasks via the processing unit.

Ciphertexts management. Traditional SSDs rely on the host-side filesystem or databases to manage data, incurring frequent communications between the host and SSDs, which in turn leads to redundant data transmissions. To counter this inefficiency, a notable solution is integrating filesystems or databases directly within the SSD [40]. Nevertheless, given that SSDs in the FL aggregator serve only as temporary data storage, integrating such sophisticated persistent data management mechanisms would be inherently inefficient. To overcome this, Flagger-CSD's FTL is tailored to directly manage ciphertexts on flash chips, thus circumventing the need for host intervention. It substitutes the traditional logical address with the *ACT-ID*, which contains data location semantics, in the logical-to-physical address mapping table. An ACT-ID is a unique 32-bit integer assigned to a ciphertext in different components of Flagger. It is comprised of three fields: **A** for different FL **A**pplications, **C** for **C**lients, and **T** for cipher**T**ext within the model updates. The ACT-ID of the first ciphertext in a flash block is used as the logical block address (LBA) of this block. The FTL ensures that ciphertexts stored in the same block are from the same client (i.e., same AC-ID), and the T-ID increments sequentially. This enables Flagger-CSD to establish a block-level mapping table from ACT-ID to Physical Block Address (PBA), discarding the original inefficient page-level mapping scheme [105] for in-storage aggregation. As such, the mapping table from ACT-ID to PBA inherently includes metadata of the ciphertexts, significantly

simplifying ciphertext management. By further modifying the NVMe commands to utilize ACT-ID instead of traditional LBA, Flagger-DPU can interact with Flagger-CSD via the NVMe protocol, which enhances coordination efficiency.

Ciphertexts aggregation. Flagger-CSD primarily takes responsibility for the combination phase tasks of ciphertexts aggregation. Upon receiving a command from Flagger-Runtime, the aggregation process within Flagger-CSD is initiated autonomously based on the ACT-ID-to-PBA mapping table without any need for host involvement. Specifically, Flagger-CSD's task scheduler in the FTL orchestrates the aggregation of ciphertexts with matched AT-IDs but different C-IDs, adding them into a partial sum in the global model update. Each MontMul unit combines two ciphertexts, generating intermediate results that are temporarily stored in the DRAM write buffer. The task scheduler then assigns new ACT-IDs to these intermediates, preparing them for the next phase of aggregation. This aggregation iteration repeats until all ciphertexts representing local updates from various clients are aggregated into a complete global model update.

Boosting aggregation efficiency. Flagger-CSD further employs multiple optimizations to streamline the combination phase within CSD. To prevent throughput degradation incurred by lengthy flash writing latencies, Flagger-CSD prioritizes the processing of intermediate ciphertexts in the DRAM buffer over those in flash chips. By adopting this prioritization strategy, Flagger-CSD fully utilizes the DRAM buffer and minimizes the likelihood of evicting intermediate results to flash chips, which is exceedingly rare in our experiments.

Given that in-storage aggregation is a read-intensive task, Flagger-CSD exploits the high bandwidth available through channel-level parallelism within the SSD. The FTL ensures that ciphertexts from the same client are distributed across different channels and that ciphertexts from different clients are interleaved, preventing read stalls on the same channel and optimizing parallelism. This strategic data layout allows the MontMul engine to access ciphertext blocks from different channels without delay, maintaining a steady flow of data to the request queue and facilitating a harmonious balance between I/O operations and aggregation computation.

D. Flagger-Runtime

Flagger-Runtime is the orchestration hub running on the host CPU, pivotal in managing and synchronizing the operations of both Flagger-DPU and Flagger-CSD. It ensures seamless interaction across different hardware components and upholds the integrity and efficiency of the FL aggregation process. The primary tasks of Flagger-Runtime are as follows:

Control plane management. Flagger-Runtime focuses on the management of control-plane tasks, including overseeing network sessions with clients and handling metadata of model updates (e.g., model sizes and public keys). Flagger-Runtime effectively communicates with Flagger-DPU by issuing session commands and synchronizing metadata through the DPU controller's exposed registers. This interaction is processed by Flagger-DPU's session manager. For Flagger-CSD, Flagger-Runtime employs the NVMe protocol, transmitting essential metadata via customized NVMe admin commands [6].

Data plane coordination. Flagger-Runtime orchestrates data transfers between Flagger-DPU and Flagger-CSD. During ciphertext collection, the orchestrator in Flagger-Runtime receives ACT-IDs of ciphertexts for transfer, generated by the DPU (cf. Section V-B). The orchestrator then initiates NVMe write commands to Flagger-CSD based on the ACT-IDs to start peer-to-peer DMA transfers between the DPU and the CSD. Similarly, ciphertexts can be dispatched using the same method.

Dynamic offloading. In scenarios where the volume of incoming traffic from clients is substantial, the compute engine on Flagger-DPU may reach capacity, necessitating an intelligent offloading mechanism. As Flagger-DPU and Flagger-CSD employ the same MontMul engine for computation (cf. Section V-A), Flagger-Runtime addresses this challenge by dynamically directing excess computation during the pre-processing phase to Flagger-CSD. This is accomplished by transferring ciphertexts directly to the CSD for processing, indicated by a reserved bit in the NVMe write command. Conversely, Flagger-CSD can also offload combination phase tasks to Flagger-DPU, with Flagger-Runtime facilitating the transfer through modified NVMe read commands. The dynamic offloading underscores the system's resilience and scalability, ensuring that computational demands are met with flexibility and efficiency. The quantitative benefits of this offloading strategy will be thoroughly examined in Section VII-C.

VI. Implementation Details of Flagger

We have developed Flagger in a real hardware environment with a full-stack software implementation. Flagger-DPU and Flagger-CSD are built on two customized FPGA boards, while Flagger-Runtime is a software implementation built on top of FATE [7], a widely used industrial FL framework.

MontMul Engines. We employed iterative digit-digit Montgomery multiplication [10], [63], an optimization of the original Montgomery algorithm for hardware implementations, to construct the MontMul unit. Our MontMul unit computes the Montgomery multiplication result of two 2048-bit integers in 1,088 cycles. Running at a clock frequency of 300MHz on both Flagger-DPU and Flagger-CSD, it offers a maximum theoretical throughput of 275.37kOPs.

Flagger-DPU. We built Flagger-DPU on a Xilinx Alveo U55C FPGA card [92], a high-performance data center acceleration card equipped with substantial computing resources. It features a high-end FPGA chip with ample resources (cf. Table IV), PCIe Gen3x16 lanes connecting with the host, 16GB memory, and two QSFP28 ports. We deployed 40 MontMul units on Flagger-DPU, enabling a maximum theoretical throughput of 11.01MOPs. We implemented a custom network stack based on EasyNet [31], an open-source 100Gbps TCP/IP stack, for direct in-network computation on TCP packets.

Flagger-CSD. We built Flagger-CSD on a Daisyplus OpenSSD [81], the most recent version of a representative

Config.	Aggregator Platform		Client Platform
	Flagger	CPU-centric	
CPU	Intel Core i7-4790, 4C8T	Intel Xeon Gold 5320, 26C52T	
OS	Ubuntu 20.04, Linux Kernel 5.4		
Mem.	16GB		128GB
Storage	Flagger-CSD 64GB × 2	Samsung 980 1TB × 2	
NIC	Flagger-DPU	Mellanox CX-5	

TABLE I: Testbed configurations.

CSD device in the OpenSSD project [8]. It employs a Xilinx ZU17EG UltraScale+ MPSoC as its processor, which contains a mid-range FPGA chip with a four-core ARM processor and 2GB DRAM. The device is linked to the host through PCIe Gen3x4 lanes as an NVMe drive. We deploy eight MontMul units on it, offering a maximum theoretical throughput of 2.20MOPs. We developed Flagger-CSD's host interface by adapting OpenExpress [38], an FPGA-based open-source NVMe controller, to the Daisyplus OpenSSD. Additionally, we implemented the FTL based on Greedy-FTL [5], and thoroughly overhauled the NVMe controller and FTL to ensure compatibility with the components on Flagger-CSD.

Flagger-Runtime. The DPU driver is created based on Xilinx Runtime (XRT) [95], the official driver for Xilinx Alveo cards. The CSD driver is built upon the Micron UNVMe library [61]. Both DPU and CSD drivers undergo substantial modifications to align with Flagger-DPU and Flagger-CSD, respectively.

VII. Evaluation

A. Methodology

Environment setup. Table I lists the testbed configurations. The aggregator and client platforms are connected via a 100Gbps DAC cable. Flagger-CSDs feature two 32GB DRAM DIMMs for storage media, as the flash chips provided by the manufacturer offer much lower bandwidth than modern SSDs [46], [98]. To simulate real NAND flash performance, we divide the DRAM into 8 channels with 75us/750us latencies for read/write operations at the page level. The DRAM access bandwidth for ARM cores in Flagger-CSD is 3.98GB/s, and the PCIe bandwidth to Flagger-CSD is 3.12GB/s, reflecting hardware metrics of real SSDs [89], [98]. Conforming real cross-silo FL environments, clients are well-provisioned with computational and network resources. Each client exploits 50 CPU threads with AVX512-IFMA52 acceleration for encryption and decryption, achieving 134.4kOPs for Paillier encryption and 324.74kOPs for decryption using 1024-bit keys, on par with cutting-edge FL accelerators [34], [103]. After local training and encryption, clients upload model updates to the aggregator at normally distributed random times, simulating real-world heterogeneous cross-silo FL scenarios [15]. Network bandwidth is capped at 1Gbps per client, with an additional 10ms to 100ms of random latency to emulate typical Internet conditions at silos [59].

Aggregator platforms. We evaluate seven aggregator platforms, including two CPU-centric baseline and five Flagger platforms: (1) `CPU-centric`, a traditional aggregator that employs a NIC, two SSDs, and 50 CPU threads with AVX512 acceleration; (2) `CPU-Mont`, similar to `CPU-centric` except executing Montgomery algorithm on CPU; (3) `FPGA-Mont`, leveraging Flagger-DPU as an FPGA-based HE accelerator without networking; (4) `CSD-acc`, employing two Flagger-CSDs for storage and in-storage aggregation, with NIC and host for networking; (5) `Flagger-raw`, a loosely coupled Flagger platform, with Flagger-DPU for network and pre/post-processing plus two Flagger-CSDs for storage and aggregation, without coordination (i.e., direct datapath, unified task scheduling and dynamic offloading between DPU and CSDs); (6) `Flagger-full`, similar to `Flagger-raw` but all the Flagger components are tightly coupled and optimized for coordination; (7) `Flagger-stream`, a streaming-based aggregator platform with Flagger-DPU for pre/post-processing and CPU to do partial aggregation during updating[2].

Abbr.	32xCN	32xLM	32xDN	32xEN	64xRN	64xVG
Network	3-layer FC [28]	LSTM [99]	DenseNet 169 [33]	EfficientNet b4 [80]	ResNet 152 [30]	VGG11 [78]
Weights	101.77K	4.02M	12M	16M	60M	138M
Client No.	32	32	32	32	64	64
Dataset	FMNIST [91]	Shakespere [1]	Cifar-10 [44]	BreakHis [106]	BreakHis [106]	Prostate2D [96]
Task	Image class.	Text gene.	Image class.	Image segment.	Image segment.	Image segment.

TABLE II: FL applications in experiments.

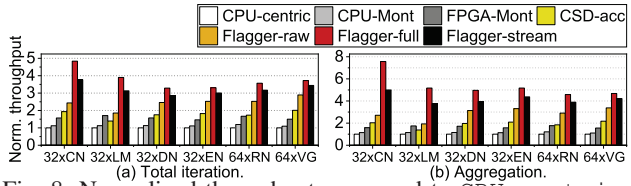


Fig. 8: Normalized throughput compared to `CPU-centric`.

Workloads. We choose six machine learning models with different sizes and application fields, following prior works [37], [101], [103]. All the workloads are prevalent in real-world FL practices [23], [51], [96]. We train them collaboratively with 32 or 64 clients and 1024-bit Paillier encryption (2048-bit modulus), as six different large-scale cross-silo FL applications. The details are listed in Table II. Due to the substantial aggregation overhead for ResNet-152 and VGG-11 with their extensive parameter sets, we use batched encryption [101] to consolidate 16 plaintexts into a single one. This method cuts down on both the computational burden and storage bloat of HE while preserving model integrity.

B. Overall Performance

Execution throughput comparison. Figures 8(a) and (b) depict the throughput of various aggregator platforms over the `CPU-centric` baseline in both total and aggregation processes in a single FL training iteration. Flagger platforms significantly surpass the baseline. Specifically, `CPU-Mont` outperforms `CPU-centric` baseline by 13% and 15% in total and aggregation cost, respectively, attributed to MontMul for HE acceleration. As MontMul is preferred to be implemented by dedicated hardware with larger bus width and efficient shift operations, `FPGA-Mont` achieves 58% and 66% in

[2]To support more complex FL mechanisms such as drop-out prevention and incentive aggregation, all the platforms, except `Flagger-stream`, store the entire model updates in the SSD before initiating the aggregation (cf. Section II-A).

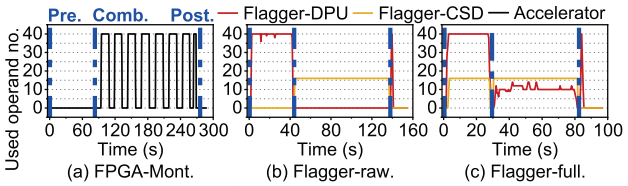


Fig. 9: Comparison of computational resources utilization.

total and aggregation speedup compared with `CPU-centric` baseline, demonstrating the potential of MontMul engine.

However, these three platforms do not employ NDP for aggregation acceleration, leaving data movement a critical performance bottleneck. Therefore, despite having 60% less computational capability, `CSD-acc` outperforms `FPGA-Mont` by 12% and 16% in aggregation and total throughput, respectively. This advantage stems from Flagger-CSD's streamlined datapath, obviating the host-side involvement for data transfers. `Flagger-raw` further enhances performance by offloading the network stack to Flagger-DPU, enabling peer-to-peer DMA and in-network pre/post-processing of ciphertexts, thus reducing Flagger-CSDs' computation load. Consequently, `Flagger-raw` achieves 190% and 145% speedup in aggregation and total throughput, respectively, when compared to baseline. Nevertheless, as `Flagger-raw` adopts a loosely coupled architecture without coordination among accelerators, it still shows sub-optimal performance. `Flagger-full` overcomes this critical issue by enabling dynamic offloading and coordination between Flagger-DPU and Flagger-CSDs. Through direct communication and unified task scheduling, `Flagger-full` exploits the potential of distinct near-data accelerators, leading to maximum speedups of 657% for aggregation and 384% for total throughput. `Flagger-stream` underperforms `Flagger-full` by 22% and 14%, underscoring Flagger's benefits, which will be discussed later.

Computational resource utilization. Figure 9 shows the computational resource usage (i.e., busy MontMul units) across different aggregator platforms. The phases `Pre`, `Comb`, and `Post` represent the time for local updates collection, ciphertext aggregation, and global update dispatching, respectively. The average utilization rates for `Flagger-full`, `Flagger-raw`, and `FPGA-Mont` stand at 52.44%, 36.56%, and 35.78% respectively. Both `Flagger-full` and `Flagger-raw` employ DPU and CSDs simultaneously, but the latter lacks dynamic task offloading between DPU and CSDs. This leads to idle periods for compute engines on Flagger-CSDs during `Pre`, and for Flagger-DPU during `Comb`. Conversely, `FPGA-Mont`'s low utilization is due to the redundant data copies. `Flagger-full` overcomes these inefficiencies with dynamic task offloading and reduced data movement, optimizing computational resource use to achieve the best performance.

Aggregator breakdown. To further analyze the cost of different aggregator platforms, Figure 10 illustrates the aggregation time breakdown across seven platforms, with times normalized against `CPU-centric`. `CPU-Mont` and `FPGA-Mont` reduce `Comb` time by 17.27% and 57.14%, respectively, showing

Accelerator	Platform	Freq.	MOPS 1024bit	MOPS 2048bit	Aggr. Time 1024bit	Aggr. Time 2048bit
PipeFL	FPGA	187MHz	1.12	-	792.89s	-
PHEP	ASIC	500MHz	-	23	-	127.24s
Flagger-DPU	FPGA	300MHz	11.01	4.09	40.00s	93.72s
Flagger-CSD	FPGA	300MHz	2.20	0.69		

TABLE III: Comparison with HE accelerators.

the efficiency of Montgomery algorithm and MontMul engine for accelerating homomorphic additions. `CSD-acc` exhibits a 46.79% higher `Pre` time due to its reliance on Flagger-CSD's limited computational resources for pre-processing. Conversely, `Flagger-raw` reduces `Pre` time by 39.93% by utilizing Flagger-DPU's in-network pre-processing capabilities. `Flagger-full` shows the greatest time reduction in `Pre` at 54.35%, attributed to offloading tasks to Flagger-CSDs. Additionally, `Flagger` significantly reduces the `Comb` phase time by 70.82%, thanks to the Flagger-CSDs and Flagger-DPU collaboration in aggregation computation. `Flagger-stream` intends to overlap `Pre` and `Comb` phases by performing streaming aggregation with CPU and pre-processing with Flagger-DPU simultaneously. Nevertheless, as the aggregation speed of CPU is much lower than the pre-processing throughput of Flagger-DPU, `Flagger-stream` still needs 86.46% longer time for `Comb` than `Pre`, thereby showing even worse aggregation performance than `Flagger-full`.

Workload impact. While Flagger achieves comparable speedup on each workload in Figure 8, `32xCN` with the smallest model (cf. Table II) gets the most benefits. Since data is uploaded to the aggregator *asynchronously* during FL aggregation, the network transmission overhead can be amortized by different uploading time slots, which is less susceptible to data volume than computational overhead (cf. Figure 10). Therefore, `32xCN` gets the most benefits from the pre/post-processing of Flagger-DPU during network transmission. For larger workloads like `32xEN`, aggregation computation overhead with Flagger-CSD becomes the major bottleneck, which can be addressed by scaling Flagger with more CSDs.

Comparison with other HE accelerators. We further compared Flagger's efficiency in enhancing FL aggregation against leading Paillier-HE accelerators (PipeFL [87] and PHEP [74], [75]) with traditional CPU-centric architecture, on `64×RN` (see Table III). We estimated their computation times for encrypting model updates based on their reported MontMul throughput, also factoring in SSD and Internal IO statistics from previous `CPU-centric` and `FPGA-Mont` evaluations. Flagger significantly outperformed PipeFL by 19.82×. Although PHEP has a 3.15× faster MontMul speed than Flagger owing to ASIC's superior performance over FPGA, Flagger still surpasses PHEP by 1.35× in overall aggregation. Furthermore, Flagger's performance could be further enhanced with ASIC implementation, as discussed in Section VII-C.

C. Dive Deep in Flagger

In this Section, we take a deep dive to analyze the performance and overheads of key components in Flagger.

In-network performance. We evaluated the network throughput between Flagger and the client both with and without the

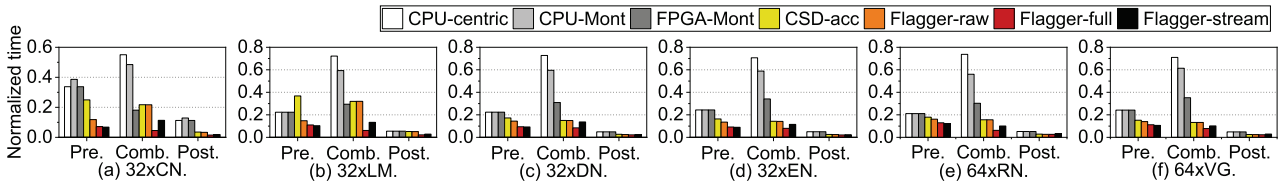


Fig. 10: Aggregator time breakdown.

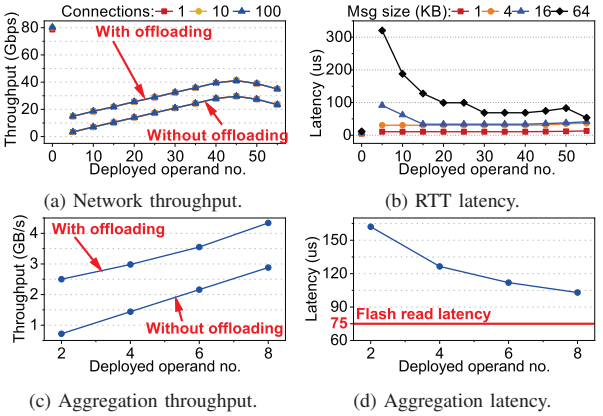


Fig. 11: In-network and in-storage performance of Flagger.

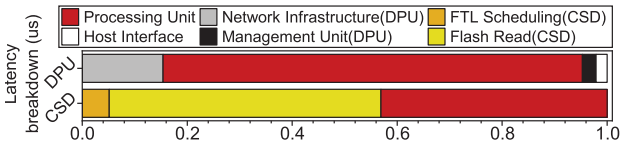


Fig. 12: Per-unit latency breakdown.

		LUT(K)	FF(K)	BRAM(Mb)	DSP
MontMul Op.		6.4	8.9	0	164
DPU	Network	141.9	152.63	27.1	0
	Proc. Unit	288.64	417.29	45.62	6560
	Mgmt. Unit	24.7	37.2	3.67	0
	Host Intf.	64.7	87.7	0.04	0
	Available	1,304	2,607	344	9024
	Percent	**39.87%**	**26.65%**	**22.22%**	**72.70%**
CSD	Proc. Unit	54.8	89.46	5.71	1312
	Host Intf.	142.78	171.42	8.57	3
	Interconnect	17.44	11.56	2.77	0
	Available	926	847	64.7	1590
	Percent	**23.22%**	**32.17%**	**26.35%**	**82.70%**

TABLE IV: FPGA resource utilization of Flagger.

dynamic offloading of pre/post-processing tasks. As depicted in Figure 11(a), sending 1GB of ciphertexts achieves a peak throughput of 79Gbps without in-network HE processing, slightly below the 100Gbps theoretical limit due to TCP packet size mismatches between ciphertexts and MSS (Section V-B). Throughput scales linearly with operand count up to 45, reaching 41.12Gbps with in-network HE processing, where Flagger-CSDs contribute 11.52Gbps. Beyond 45 operands, throughput declines due to FPGA resource over-utilization, which imposes challenges on FPGA synthesis and engine frequency. Connection count doesn't impact throughput, showing Flagger's multi-connection efficiency in cross-silo FL.

Figure 11(b) presents the round-trip time (RTT) between Flagger and clients with varying message sizes and operands on Flagger-DPU. After removing the extra latency (Section VII-A), we found that larger messages increase latency due to heavier pre/post-processing loads, while additional computational resources effectively mitigate the overheads.

In-storage performance. Figure 11(c) presents Flagger's aggregation throughput with varying operand counts on a single Flagger-CSD, both with and without dynamic task offloading to Flagger-DPU. We aggregated 64GB of ciphertext from Flagger-CSDs into 1GB, mirroring typical FL scenarios. Without dynamic offloading, the throughput increases linearly with the number of operands, echoing the network throughput trend in Figure 11(a). This indicates that Flagger-CSD's limited computational capacity becomes the primary bottleneck. Flagger mitigates this by partially offloading to Flagger-DPU, which yields an average of 1.54GB/s higher throughput. Figure 11(d) illustrates the latency for aggregating two ciphertext pages in relation to operand count. At full resource utilization (i.e., 8 operands), aggregation only incurs 35.07% additional latency compared to flash page read latency, showcasing Flagger-CSD's efficient in-storage aggregation.

Per-unit breakdown. We analyzed the latency breakdown for Flagger-DPU and Flagger-CSD, as depicted in Figure 12. For Flagger-DPU, pre-processing a 64KB TCP message with 40 MontMul units (see Figure 11(b)) primarily consumes time in converting ciphertexts to Montgomery space, accounting for 79.87% of the total latency. This indicates that limited computational resources are the main constraint for Flagger-DPU, which could be improved with ASIC implementation. For Flagger-CSD, we analyzed the latency when concurrently aggregating eight pages of ciphertexts across eight flash channels. Flash read operations take up 51.72% of execution time, similar to the sum time of processing unit computation and FTL scheduling. Based on the observation above, we have built the computation-IO pipeline which effectively conceals the NAND flash reading latency during aggregation tasks.

Choices for pre-processing. To further demonstrate the efficiency of in-network pre-processing in Flagger, we compare `Flagger-full` with `Client-pre`, which relocates pre-processing tasks to the client side. Figure 13 provides a time cost breakdown for both `Flagger-full` and

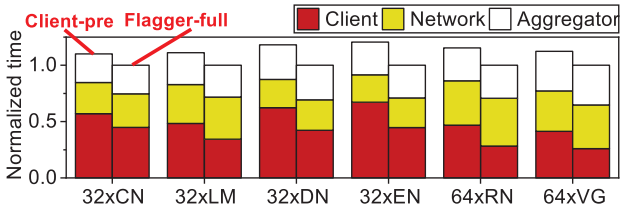
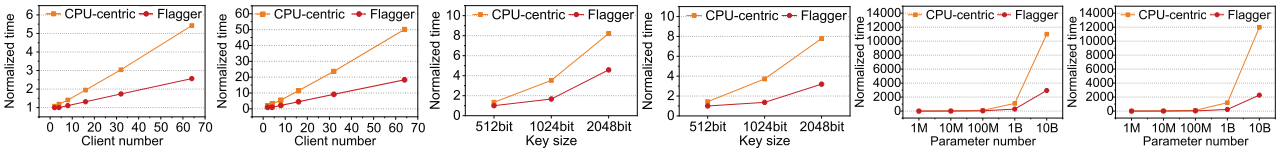


Fig. 13: Total time cost breakdown.



(a) Total cost. (b) Aggregation cost. (c) Total cost. (d) Aggregation cost. (e) Total cost. (f) Aggregation cost.

Fig. 14: Training time of LM with varying configurations.

Client-pre. Here, Client refers to the client-side time cost, encompassing encryption, decryption, and pre-processing in Client-pre. Network denotes the time cost associated with network transmission, including in-network pre-processing in Flagger-full. Aggregator represents the time cost on the aggregator side, covering both aggregation and the post-processing phase.

Our observations reveal that Flagger-full reduces the total time cost by up to 20.56% compared to Client-pre. This is because, in the context of Client-pre, clients bear the burden of encryption tasks, which cannot allocate spare computing resources for pre-processing. In addition, due to the lack of dedicated hardware (i.e., DPU) for accelerating and overlapping pre-processing with network transmission, the efficiency of client-side pre-processing is significantly diminished. As a result, Client in Client-pre experiences an average increase in the total time cost by 17.12%. In contrast, the fully-pipelined in-network pre-processing of Flagger-full minorly prolongs the Network cost, leading to a mere 2.49% longer total time cost compared to Network of Client-pre. This further underscores the efficiency of Flagger. Note that pre-processing does not impact the ciphertext size. This is because the Paillier-based HE employs a modulus of the same size as the ciphertext.

Resource consumption. Table IV shows the FPGA resource utilization of a single MontMul unit, Flagger-DPU, and Flagger-CSD. The placement and routing are completed with Vivado and Vitis 2021.2 [93]. The processing units containing MontMul units rely heavily on DSP slices to accelerate homomorphic computations, which is the primary limitation of the maximum available MontMul units on both Flagger-DPU and Flagger-CSD. Flagger-DPU and Flagger-CSD exploit 72.7% and 82.7% DSP resources on the FPGA, respectively, leaving space for further extension of acceleration functions.

ASIC performance assessment. We further explore the potential of ASIC implementation for Flagger, given its constraints on operating frequency and DSP resources in FPGA-based prototypes. We synthesize the Flagger's processing unit using Synopsys Design Compiler and the FreePDK 45nm library [79]. Following prior work [103], we allocate a die area of $130mm^2$, enabling space for 150 MontMul units on the chip. Our evaluation results reveal that the ASIC implementation achieves a throughput of 76.79 MOPs at 600 MHz, which is nearly 7× the performance of its FPGA counterpart.

D. Scalability Tests

Number of clients. Figure 14(a,b) show the total time cost and the aggregator cost of CPU-centric and Flagger with varying numbers of clients training LM collaboratively. The results are normalized to that of Flagger with 2 clients. Intuitively, more clients impose larger pressure on the aggregator, as analyzed in Section III. Although both platforms scale almost linearly with more clients, Flagger shows better scalability than CPU-centric. This demonstrates that Flagger is more suitable for larger-scale cross-silo FL settings.

HE key size. Similarly, we vary the key size of HE for encrypting model updates in training LM with 64 clients, as shown in Figure 14(c,d). The results are normalized to that of Flagger with 512-bit key encryption. Although longer HE keys incur larger computation and storage overheads, Flagger is less affected than CPU-centric, demonstrating that Flagger is better suited for stricter privacy requirements.

Model size. Based on our evaluation statistics, we conduct a theoretical analysis of Flagger's performance by scaling to larger model sizes and 64 clients. Figure 14(e,f) shows the execution time normalized to CPU-centric baseline with 1M model. Compared to CPU-centric, Flagger can reap more benefits when scaling to larger models, demonstrating the potential benefits of Flagger to accelerate emerging large-scale workloads such as large-language models.

VIII. Related Work and Discussion

Accelerating FL. Multiple studies [21], [87], [97], [103] have utilized FPGA or GPU accelerators to enhance Paillier HE computation in FL, while Intel has optimized HE using single-instruction-multiple-data (SIMD) on its CPUs with AVX512 instruction sets [34]. These approaches, however, primarily benefit FL clients and do not address the aggregator's challenges, including network, storage, and datapath overheads. Techniques like BatchCrypt [101] focus on batching plaintexts to reduce ciphertext size, which is compatible with Flagger as it can process batched ciphertexts seamlessly.

FL aggregator architecture. Multiple prior works explore new aggregator architectures to mitigate FL aggregation overheads. [18], [20], [76], [100] leverage programmable network devices to allow in-network aggregation for FL without HE, which is unsuitable for cross-silo scenarios. Recently, some works [36], [41], [86] explore deploying multiple aggregators to allow distributed aggregation in cross-device FL. These solutions are possible to be extended to cross-silo FL and combined with Flagger in the future.

Near-data processing. NDP has been an emerging research topic on accelerating computation by minimizing data movement. Nevertheless, few prior studies integrate distinct NDP accelerators to enhance performance. In networking, works

like [42], [62], [73] offload TCP processing to DPUs, lightening the host's load but still depending on it. ATP [48] employs in-network aggregation on switches for distributed deep learning, which is however incompatible with cross-silo FL aggregation due to the intensive HE computation and complex WAN environment. Flagger-DPU implements network stacks on DPUs, facilitating in-network computations on TCP packets and direct DPU-CSD communications. In storage, prior research [12], [47], [52], [58] uses CSDs to speed up applications via in-storage processing. Flagger-CSD similarly minimizes aggregation overheads of ciphertexts in cross-silo FL by employing in-storage computation and facilitates self-governing data handling, eliminating host intervention. Flagger further merges DPU and CSD capabilities to significantly quicken cross-silo FL aggregation, reducing FL training time.

Adaptability of Flagger. Other HE schemes, such as FHE, face similar challenges in network, storage, and computation as Paillier-based HE when applied to FL. Although Flagger is designed for Paillier-based FL, the collaborative near-data acceleration approach in Flagger remains relevant under the substantial demands of other HE-based FL. Adjustments for different HE algorithms are confined to the HE operands. Moreover, Flagger's use of reconfigurable hardware in both DPU and CSD enhances its adaptability for deploying new operands across HE algorithms, such as substituting MontMul with NTT operands [70] for FHE schemes to facilitate pre-/post-processing on Flagger-DPU. On the other hand, unlike Paillier-based FL, alternative lightweight privacy-preserving techniques (e.g., differential privacy [88]) exhibit lesser demands on aggregators' storage and computing capacities. They, thus, cannot reap the full benefits from Flagger, which is designed to address storage and computing issues. Due to the lower impact on model accuracy and higher security assurance [24], cross-silo FL prefers HE rather than lightweight ones [103], prompting our focus on HE performance optimization.

Impact of Flagger. Mitigating the aggregation bottleneck in cross-silo FL often requires extensive DRAM resources and computing power, leading to increased costs. Flagger presents a cost-effective alternative by using NAND flash instead of expensive DRAM, along with DPU and CSD integration to boost aggregation performance. This setup significantly exceeds the performance of the conventional CPU-centric aggregators with lower cost, which paves the way for widespread adoption of cross-silo FL in various practical domains [23], [29], [96].

IX. Conclusion

We observe that the aggregator has become the key bottleneck in large-scale cross-silo FL due to its architectural inefficiency. To address this, we propose Flagger, an efficient and high-performance aggregator architecture, which leverages both DPU and CSD as two distinct NDP accelerators to cooperatively boost FL aggregation. Flagger exploits in-network processing and in-storage aggregation simultaneously, with a host runtime to orchestrate the accelerators and enable peer-to-peer DMA between DPU and CSD. Our evaluation shows that Flagger reduces aggregation time cost by 436% on average, compared with traditional CPU-centric aggregator.

Acknowledgment

We thank the anonymous shepherd and reviewers for their insightful comments and feedback. This work is mainly supported by the National Key Research and Development Program of China under Grant No. 2023YFB4502702, the National Natural Science Foundation of China under Grant No. 62332021, and the Fundamental Research Funds for the Central Universities, Peking University. Dr. Liang is supported in part by the National Natural Science Foundation of China under Grant No. 62202453. Dr. Mao is supported in part by the National Natural Science Foundation of China under Grant No. U22A2027 and No. 61972325. Dr. Mingzhe Zhang is supported in part by the Strategic Priority Research Program of the Chinese Academy of Sciences under Grant No. XDB44030200. Dr. Jung is supported in part by NRF 2021R1A2C4001773, IITP 2021-0-00524, IITP tel:2022-0-00117, IITP RS-2023-00221040, G01230749, SRFC-IT2302-05, Samsung HiPER (G01220296), and KAIST IDEC. Jie Zhang is affiliated with School of Computer Science at Peking University and Zhongguancun Laboratory, and is the corresponding author.

References

[1] "Shakespere dataset," https://www.tensorflow.org/text/tutorials/text_generation.

[2] "Regulation (eu) 2016/679 of the european parliament and of the council of 27 april 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing directive 95/46/ec (general data protection regulation)." https://eur-lex.europa.eu/eli/reg/2016/679/oj, 2016.

[3] "Cybersecurity law of the people's republic of china." http://www.lawinfochina.com/display.aspx?id=22826&lib=law, 2017.

[4] "California consumer privacy act (ccpa)." https://oag.ca.gov/privacy/ccpa, 2018.

[5] "Greedy-ftl," https://github.com/Cosmos-OpenSSD/Cosmos-plus-OpenSSD/tree/master/source/software/GreedyFTL-3.0.0, 2019.

[6] "Nvm express nvm command set specification 1.0c," https://nvmexpress.org/wp-content/uploads/NVM-Express-NVM-Command-Set-Specification-1.0c-2022.10.03-Ratified.pdf, 2022.

[7] "FATE (federated ai technology enabler)," https://github.com/FederatedAI/FATE, 2023.

[8] "The openssd project," http://www.openssd-project.org/, 2023.

[9] M. Altman, A. Wood, D. R. O'Brien, S. Vadhan, and U. Gasser, "Towards a modern approach to privacy-aware government data releases," *Berkeley Technology Law Journal*, vol. 30, no. 3, pp. 1967–2072, 2015.

[10] D. Amiet, A. Curiger, and P. Zbinden, "Flexible fpga-based architectures for curve point multiplication over gf (p)," in *2016 Euromicro Conference on Digital System Design (DSD)*. IEEE, 2016, pp. 107–114.

[11] Y. Aono, T. Hayashi, L. Wang, S. Moriai *et al.*, "Privacy-preserving deep learning via additively homomorphic encryption," *IEEE transactions on information forensics and security*, vol. 13, no. 5, pp. 1333–1345, 2017.

[12] J. Bae, J. Lee, Y. Jin, S. Son, S. Kim, H. Jang, T. J. Ham, and J. W. Lee, "{FlashNeuron}:{SSD-Enabled}{Large-Batch} training of very deep neural networks," in *19th USENIX Conference on File and Storage Technologies (FAST 21)*, 2021, pp. 387–401.

[13] R. Balasubramonian, J. Chang, T. Manning, J. H. Moreno, R. Murphy, R. Nair, and S. Swanson, "Near-data processing: Insights from a micro-46 workshop," *IEEE Micro*, vol. 34, no. 4, pp. 36–42, 2014.

[14] F. Boemer, S. Kim, G. Seifu, F. DM de Souza, and V. Gopal, "Intel hexl: Accelerating homomorphic encryption with intel avx512-ifma52," in *Proceedings of the 9th on Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, 2021, pp. 57–62.

[15] R. C. Brum, P. Sens, L. Arantes, M. C. Castro, and L. M. de A. Drummond, "Optimizing execution time and costs of cross-silo federated learning applications with datasets on different cloud providers," in *2022 IEEE 34th International Symposium on Computer Architecture and High Performance Computing (SBAC-PAD)*, 2022, pp. 253–262.

[16] M. S. Brunella, M. Bonola, and A. Trivedi, "Hyperion: A case for unified, self-hosting, zero-cpu data-processing units (dpus)," *arXiv preprint arXiv:2205.08882*, 2022.

[17] I. Burstein, "Nvidia data center processing unit (dpu) architecture," in *2021 IEEE Hot Chips 33 Symposium (HCS)*. IEEE, 2021, pp. 1–20.

[18] F. Chen, P. Li, and T. Miyazaki, "In-network aggregation for privacy-preserving federated learning," in *2021 International Conference on Information and Communication Technologies for Disaster Management (ICT-DM)*. IEEE, 2021, pp. 49–56.

[19] F. Chen, D. A. Koufaty, and X. Zhang, "Understanding intrinsic characteristics and system implications of flash memory based solid state drives," *ACM SIGMETRICS Performance Evaluation Review*, vol. 37, no. 1, pp. 181–192, 2009.

[20] X. Chen, G. Zhu, Y. Deng, and Y. Fang, "Federated learning over multihop wireless networks with in-network aggregation," *IEEE Transactions on Wireless Communications*, vol. 21, no. 6, pp. 4622–4634, 2022.

[21] X. Cheng, W. Lu, X. Huang, S. Hu, and K. Chen, "Haflo: Gpu-based acceleration for federated logistic regression," *arXiv preprint arXiv:2107.13797*, 2021.

[22] J. Dastidar, "Fpgas and their evolving role in domain specific architectures: A case study of the amd 400g adaptive smartnic/dpu soc," in *Proceedings of the 2023 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, 2023, pp. 151–151.

[23] A. Durrant, M. Markovic, D. Matthews, D. May, J. Enright, and G. Leontidis, "The role of cross-silo federated learning in facilitating data sharing in the agri-food sector," *Computers and Electronics in Agriculture*, vol. 193, p. 106648, 2022.

[24] A. El Ouadrhiri and A. Abdelhadi, "Differential privacy for deep and federated learning: A survey," *IEEE access*, vol. 10, pp. 22 359–22 380, 2022.

[25] S. E. Eldridge and C. D. Walter, "Hardware implementation of montgomery's modular multiplication algorithm," *IEEE transactions on Computers*, vol. 42, no. 6, pp. 693–699, 1993.

[26] L. Gao, L. Li, Y. Chen, W. Zheng, C. Xu, and M. Xu, "Fifl: A fair incentive mechanism for federated learning," in *Proceedings of the 50th International Conference on Parallel Processing*, 2021, pp. 1–10.

[27] B. Gu, A. S. Yoon, D.-H. Bae, I. Jo, J. Lee, J. Yoon, J.-U. Kang, M. Kwon, C. Yoon, S. Cho *et al.*, "Biscuit: A framework for near-data processing of big data workloads," *ACM SIGARCH Computer Architecture News*, vol. 44, no. 3, pp. 153–165, 2016.

[28] J. Gu, Z. Wang, J. Kuen, L. Ma, A. Shahroudy, B. Shuai, T. Liu, X. Wang, G. Wang, J. Cai *et al.*, "Recent advances in convolutional neural networks," *Pattern recognition*, vol. 77, pp. 354–377, 2018.

[29] E. Guberović, C. Alexopoulos, I. Bosnić, and I. Čavrak, "Framework for federated learning open models in e-government applications," *Interdisciplinary Description of Complex Systems: INDECS*, vol. 20, no. 2, pp. 162–177, 2022.

[30] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.

[31] Z. He, D. Korolija, and G. Alonso, "Easynet: 100 gbps network for hls," in *2021 31st International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE, 2021, pp. 197–203.

[32] C. Huang, J. Huang, and X. Liu, "Cross-silo federated learning: Challenges and opportunities," *arXiv preprint arXiv:2206.12949*, 2022.

[33] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 4700–4708.

[34] Intel, "Intel paillier cryptosystem library," https://github.com/intel/pailliercryptolib, 2022.

[35] Intel, "Data plane development kit," https://www.dpdk.org/, 2023.

[36] K. Jayaram, V. Muthusamy, G. Thomas, A. Verma, and M. Purcell, "Lambda fl: Serverless aggregation for federated learning," in *International Workshop on Trustable, Verifiable and Auditable Federated Learning*, 2022, p. 9.

[37] Z. Jiang, W. Wang, and Y. Liu, "Flashe: Additively symmetric homomorphic encryption for cross-silo federated learning," *arXiv preprint arXiv:2109.00675*, 2021.

[38] M. Jung, "{OpenExpress}: Fully hardware automated open research framework for future fast {NVMe} devices," in *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, 2020, pp. 649–656.

[39] P. Kairouz, H. B. McMahan, B. Avent, A. Bellet, M. Bennis, A. N. Bhagoji, K. Bonawitz, Z. Charles, G. Cormode, R. Cummings *et al.*, "Advances and open problems in federated learning," *Foundations and Trends® in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021.

[40] S. Kannan, A. C. Arpaci-Dusseau, R. H. Arpaci-Dusseau, Y. Wang, J. Xu, and G. Palani, "Designing a true {Direct-Access} file system with {DevFS}," in *16th USENIX Conference on File and Storage Technologies (FAST 18)*, 2018, pp. 241–256.

[41] A. Khan, Y. Li, A. Anwar, Y. Cheng, T. Hoang, N. Baracaldo, and A. Butt, "A distributed and elastic aggregation service for scalable federated learning systems," *arXiv preprint arXiv:2204.07767*, 2022.

[42] T. Kim, D. M. Ng, J. Gong, Y. Kwon, M. Yu, and K. Park, "Rearchitecting the {TCP} stack for {I/O-Offloaded} content delivery," in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, 2023, pp. 275–292.

[43] G. Koo, K. K. Matam, T. I, H. K. G. Narra, J. Li, H.-W. Tseng, S. Swanson, and M. Annavaram, "Summarizer: trading communication with computing near storage," in *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture*, 2017, pp. 219–231.

[44] A. Krizhevsky, G. Hinton *et al.*, "Learning multiple layers of features from tiny images," 2009.

[45] S. Kumar, D. E. Culler, and R. A. Popa, "Mage: Nearly zero-cost virtual memory for secure computation." in *OSDI*, 2021, pp. 367–385.

[46] J. Kwak, S. Lee, K. Park, J. Jeong, and Y. H. Song, "Cosmos+ openssd: Rapid prototype for flash storage systems," *ACM Transactions on Storage (TOS)*, vol. 16, no. 3, pp. 1–35, 2020.

[47] M. Kwon, D. Gouk, S. Lee, and M. Jung, "{Hardware/Software}{Co-Programmable} framework for computational {SSDs} to accelerate deep learning service on {Large-Scale} graphs," in *20th USENIX Conference on File and Storage Technologies (FAST 22)*, 2022, pp. 147–164.

[48] C. Lao, Y. Le, K. Mahajan, Y. Chen, W. Wu, A. Akella, and M. Swift, "{ATP}: In-network aggregation for multi-tenant learning," in *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*, 2021, pp. 741–761.

[49] J. H. Lee, H. Zhang, V. Lagrange, P. Krishnamoorthy, X. Zhao, and Y. S. Ki, "Smartssd: Fpga accelerated near-storage data analytics on ssd," *IEEE Computer architecture letters*, vol. 19, no. 2, pp. 110–113, 2020.

[50] Y. Lee, J. Chung, and M. Rhu, "Smartsage: Training large-scale graph neural networks using in-storage processing architectures," in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, 2022, pp. 932–945.

[51] L. Li, N. Xie, and S. Yuan, "A federated learning framework for breast cancer histopathological image classification," *Electronics*, vol. 11, no. 22, p. 3767, 2022.

[52] J. Lin, L. Liang, Z. Qu, I. Ahmad, L. Liu, F. Tu, T. Gupta, Y. Ding, and Y. Xie, "Inspire: in-s torage p rivate i nformation re trieval via protocol and architecture co-design," in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, 2022, pp. 102–115.

[53] J. Liu, C. Maltzahn, C. Ulmer, and M. L. Curry, "Performance characteristics of the bluefield-2 smartnic," *arXiv preprint arXiv:2105.06619*, 2021.

[54] Z. Liu, J. Guo, W. Yang, J. Fan, K.-Y. Lam, and J. Zhao, "Privacy-preserving aggregation in federated learning: A survey," *IEEE Transactions on Big Data*, 2022.

[55] S. K. Lo, Q. Lu, C. Wang, H.-Y. Paik, and L. Zhu, "A systematic literature review on federated machine learning: From a software engineering perspective," *ACM Computing Surveys (CSUR)*, vol. 54, no. 5, pp. 1–39, 2021.

[56] S. K. Lo, Q. Lu, L. Zhu, H.-Y. Paik, X. Xu, and C. Wang, "Architectural patterns for the design of federated learning systems," *Journal of Systems and Software*, vol. 191, p. 111357, 2022.

[57] F. Luo, S. Al-Kuwari, and Y. Ding, "Svfl: Efficient secure aggregation and verification for cross-silo federated learning," *IEEE Transactions on Mobile Computing*, 2022.

[58] N. Mansouri Ghiasi, J. Park, H. Mustafa, J. Kim, A. Olgun, A. Gollwitzer, D. Senol Cali, C. Firtina, H. Mao, N. Almadhoun Alserr *et al.*, "Genstore: a high-performance in-storage processing system for genome sequence analysis," in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2022, pp. 635–654.

[59] O. Marfoq, C. Xu, G. Neglia, and R. Vidal, "Throughput-optimal topology design for cross-silo federated learning," *Advances in Neural Information Processing Systems*, vol. 33, pp. 19 478–19 487, 2020.

[60] R. Miao, L. Zhu, S. Ma, K. Qian, S. Zhuang, B. Li, S. Cheng, J. Gao, Y. Zhuang, P. Zhang *et al.*, "From luna to solar: the evolutions of the compute-to-storage networks in alibaba cloud," in *Proceedings of the ACM SIGCOMM 2022 Conference*, 2022, pp. 753–766.

[61] Micron, "Unvme - a user space nvme driver project," https://github.com/zenglg/unvme, 2015.

[62] Y. Moon, S. Lee, M. A. Jamshed, and K. Park, "{AccelTCP}: Accelerating network applications with stateful {TCP} offloading," in *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, 2020, pp. 77–92.

[63] M. Morales-Sandoval and A. Diaz-Perez, "Scalable gf (p) montgomery multiplier based on a digit–digit computation approach," *IET Computers & Digital Techniques*, vol. 10, no. 3, pp. 102–109, 2016.

[64] M. Necker, D. Contis, and D. Schimmel, "Tcp-stream reassembly and state tracking in hardware," in *Proceedings. 10th Annual IEEE Symposium on Field-Programmable Custom Computing Machines*. IEEE, 2002, pp. 286–287.

[65] P. Paillier, "Public-key cryptosystems based on composite degree residuosity classes," in *International conference on the theory and applications of cryptographic techniques*. Springer, 1999, pp. 223–238.

[66] J. Park, N. Y. Yu, and H. Lim, "Privacy-preserving federated learning using homomorphic encryption with different encryption keys," in *2022 13th International Conference on Information and Communication Technology Convergence (ICTC)*. IEEE, 2022, pp. 1869–1871.

[67] I. Pejic, R. Wang, and K. Liang, "Effect of homomorphic encryption on the performance of training federated learning generative adversarial networks," *arXiv preprint arXiv:2207.00263*, 2022.

[68] J. Postel, "The tcp maximum segment size and related topics," Tech. Rep., 1983.

[69] Z. Ruan, T. He, and J. Cong, "{INSIDER}: Designing {In-Storage} computing system for emerging {High-Performance} drive," in *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, 2019, pp. 379–394.

[70] N. Samardzic, A. Feldmann, A. Krastev, S. Devadas, R. Dreslinski, C. Peikert, and D. Sanchez, "F1: A fast and programmable accelerator for fully homomorphic encryption," in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021, pp. 238–252.

[71] B. Santos, J. Templeton, R. Chemli, S. Molladavoudi, F. Pugliese, E. Cerasti, M. De Cubellis, and M. Jug, "Insights into privacy-preserving federated ma-chine learning from the perspective of a national statistical office," 2023.

[72] J. Shao, Y. Sun, S. Li, and J. Zhang, "Dres-fl: Dropout-resilient secure federated learning for non-iid clients via secret data sharing," *Advances in Neural Information Processing Systems*, vol. 35, pp. 10 533–10 545, 2022.

[73] R. Shashidhara, T. Stamler, A. Kaufmann, and S. Peter, "{FlexTOE}: Flexible {TCP} offload with {Fine-Grained} parallelism," in *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, 2022, pp. 87–102.

[74] G. Shi, Y. Li, X. Wang, Z. Tan, D. Cao, J. Cai, Y. Wei, Z. Li, W. Zhang, Y. Wu *et al.*, "Phep: Paillier homomorphic encryption processors for privacy-preserving applications in cloud computing," in *2023 IEEE Hot Chips 35 Symposium (HCS)*. IEEE, 2023, pp. 1–20.

[75] G. Shi, Z. Tan, D. Cao, J. Cai, W. Zhang, Y. Wu, and K. Ma, "A 28nm 68mops 0.18\mu\mathrm {J}/\text {Op} paillier homomorphic encryption processor with bit-serial sparse ciphertext computing," in *2023 IEEE International Solid-State Circuits Conference (ISSCC)*. IEEE, 2023, pp. 242–244.

[76] N. Shibahara, M. Koibuchi, and H. Matsutani, "A case for offloading federated learning server on smart nic," *arXiv preprint arXiv:2307.06561*, 2023.

[77] D. Sidler, Z. István, and G. Alonso, "Low-latency tcp/ip stack for data center applications," in *2016 26th International Conference on Field Programmable Logic and Applications (FPL)*. IEEE, 2016, pp. 1–4.

[78] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *arXiv preprint arXiv:1409.1556*, 2014.

[79] J. E. Stine, I. Castellanos, M. Wood, J. Henson, F. Love, W. R. Davis, P. D. Franzon, M. Bucher, S. Basavarajaiah, J. Oh *et al.*, "Freepdk: An open-source variation-aware design kit," in *2007 IEEE international conference on Microelectronic Systems Education (MSE'07)*. IEEE, 2007, pp. 173–174.

[80] M. Tan and Q. Le, "Efficientnet: Rethinking model scaling for convolutional neural networks," in *International conference on machine learning*. PMLR, 2019, pp. 6105–6114.

[81] C. Technology, "Daisyplus pcie nvme storage accelerator," https://www.crz-tech.com/crz/article/DaisyPlus/, 2021.

[82] Y. Tian, Y. Wan, L. Lyu, D. Yao, H. Jin, and L. Sun, "Fedbert: when federated learning meets pre-training," *ACM Transactions on Intelligent Systems and Technology (TIST)*, vol. 13, no. 4, pp. 1–26, 2022.

[83] M. Tork, L. Maudlej, and M. Silberstein, "Lynx: A smartnic-driven accelerator-centric architecture for network servers," in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2020, pp. 117–131.

[84] H. Wang, S. Ma, H.-N. Dai, M. Imran, and T. Wang, "Blockchain-based data privacy management with nudge theory in open banking," *Future Generation Computer Systems*, vol. 110, pp. 812–823, 2020.

[85] Z. Wang, H. Huang, J. Zhang, F. Wu, and G. Alonso, "{FpgaNIC}: An {FPGA-based} versatile 100gb {SmartNIC} for {GPUs}," in *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, 2022, pp. 967–986.

[86] Z. Wang, H. Xu, J. Liu, H. Huang, C. Qiao, and Y. Zhao, "Resource-efficient federated learning with hierarchical aggregation in edge computing," in *IEEE INFOCOM 2021-IEEE Conference on Computer Communications*. IEEE, 2021, pp. 1–10.

[87] Z. Wang, B. Che, L. Guo, Y. Du, Y. Chen, J. Zhao, and W. He, "Pipefl: Hardware/software co-design of an fpga accelerator for federated learning," *IEEE Access*, vol. 10, pp. 98 649–98 661, 2022.

[88] K. Wei, J. Li, M. Ding, C. Ma, H. H. Yang, F. Farokhi, S. Jin, T. Q. Quek, and H. V. Poor, "Federated learning with differential privacy: Algorithms and performance analysis," *IEEE Transactions on Information Forensics and Security*, vol. 15, pp. 3454–3469, 2020.

[89] M. Wilkening, U. Gupta, S. Hsia, C. Trippel, C.-J. Wu, D. Brooks, and G.-Y. Wei, "Recssd: near data processing for solid state drive based recommendation inference," in *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2021, pp. 717–729.

[90] G. Wu, P. Huang, and X. He, "Reducing ssd access latency via nand flash program and erase suspension," *Journal of Systems Architecture*, vol. 60, no. 4, pp. 345–356, 2014.

[91] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms," *arXiv preprint arXiv:1708.07747*, 2017.

[92] Xilinx, "Alveo u55c high performance compute card," https://www.xilinx.com/products/boards-and-kits/alveo/u55c.html, 2021.

[93] Xilinx, "Vitis software platform release notes," https://docs.xilinx.com/r/2021.2-English/ug1393-vitis-application-acceleration/Vitis-Software-Platform-Release-Notes, 2021.

[94] Xilinx, "Ultrascale+ integrated 100g ethernet subsystem," https://www.xilinx.com/products/intellectual-property/cmac_usplus.html#documentation, 2023.

[95] Xilinx, "Xilinx runtime library (xrt)," https://www.xilinx.com/products/design-tools/vitis/xrt.html, 2023.

[96] A. Xu, W. Li, P. Guo, D. Yang, H. R. Roth, A. Hatamizadeh, C. Zhao, D. Xu, H. Huang, and Z. Xu, "Closing the generalization gap of cross-silo federated medical image segmentation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 20 866–20 875.

[97] Z. Yang, S. Hu, and K. Chen, "Fpga-based hardware accelerator of homomorphic encryption for efficient federated learning," *arXiv preprint arXiv:2007.10560*, 2020.

[98] Z. Yang, Y. Lu, X. Liao, Y. Chen, J. Li, S. He, and J. Shu, "{λ-IO}: A unified {IO} stack for computational storage," in *21st USENIX*

Conference on File and Storage Technologies (FAST 23), 2023, pp. 347–362.

[99] Y. Yu, X. Si, C. Hu, and J. Zhang, "A review of recurrent neural networks: Lstm cells and network architectures," *Neural computation*, vol. 31, no. 7, pp. 1235–1270, 2019.

[100] S. Zang, J. Fei, X. Ren, Y. Wang, Z. Cao, and J. Wu, "A smartnic-based secure aggregation scheme for federated learning," 2022.

[101] C. Zhang, S. Li, J. Xia, W. Wang, F. Yan, and Y. Liu, "Batchcrypt: Efficient homomorphic encryption for cross-silo federated learning," in *Proceedings of the 2020 USENIX Annual Technical Conference (USENIX ATC 2020)*, 2020.

[102] J. Zhang and M. Jung, "Zng: Architecting gpu multi-processors with new flash for scalable data analysis," in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2020, pp. 1064–1075.

[103] J. Zhang, X. Cheng, W. Wang, L. Yang, J. Hu, and K. Chen, "{FLASH}: Towards a high-performance hardware acceleration architecture for cross-silo federated learning," in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, 2023, pp. 1057–1079.

[104] L. Zhou, S. Pan, J. Wang, and A. V. Vasilakos, "Machine learning on big data: Opportunities and challenges," *Neurocomputing*, vol. 237, pp. 350–361, 2017.

[105] Y. Zhou, F. Wu, P. Huang, X. He, C. Xie, and J. Zhou, "An efficient page-level ftl to optimize address translation in flash memory," in *Proceedings of the Tenth European Conference on Computer Systems*, 2015, pp. 1–16.

[106] C. Zhu, F. Song, Y. Wang, H. Dong, Y. Guo, and J. Liu, "Breast cancer histopathology image classification through assembling multiple compact cnns," *BMC medical informatics and decision making*, vol. 19, no. 1, pp. 1–17, 2019.